Respostas tbl 3

EXERCICIO 1 =

```
const express = require('express');
const cors = require('cors');

const app = express();

app.use(cors());

app.get('/usuarios', (req, res) => {
  res.json([{ id: 1, nome: 'Ana' }]);
});

app.listen(3000, () => {
  console.log('Servidor rodando na porta 3000');
});
```

trecho que habilita o cors

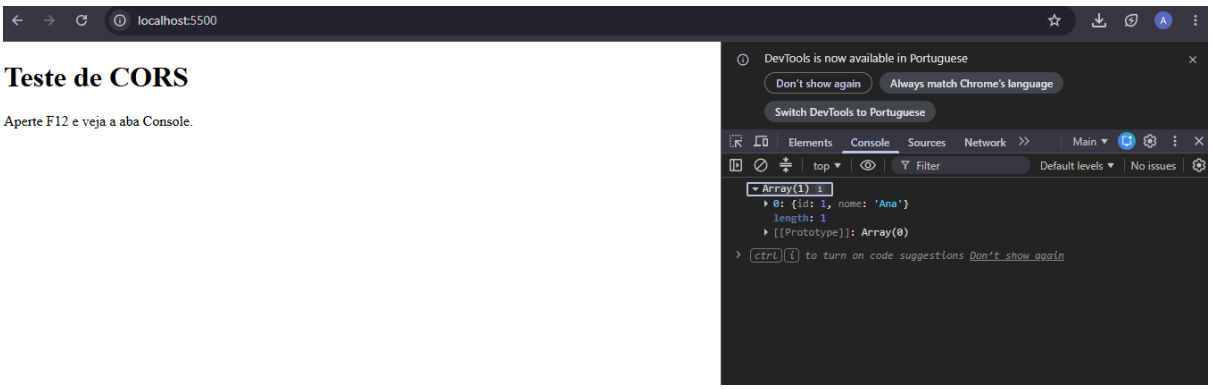
EXERCICIO 2 =

Porque **quem aplica a regra do CORS é o navegador**, não o servidor. O navegador é "desconfiado": se uma página de um endereço tenta ler dados de outro endereço, ele só deixa se o servidor der permissão.

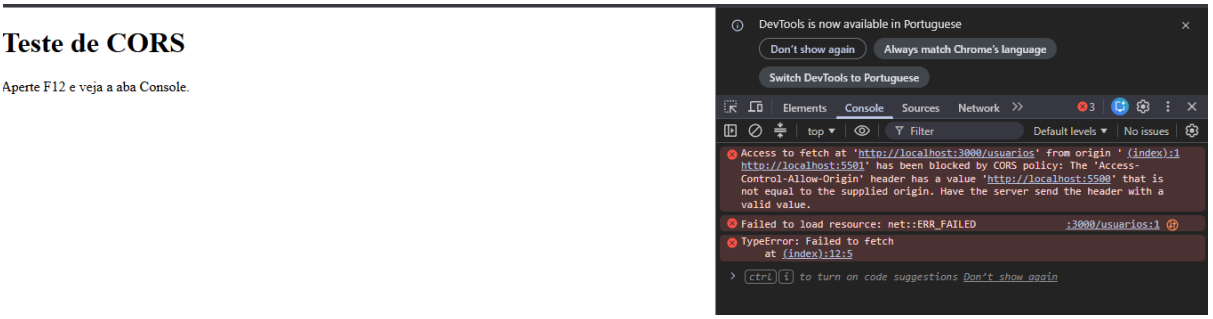
O Postman não é um navegador, então ele simplesmente não faz essa checagem. Ele pede e recebe a resposta, sem perguntar nada.

EXERCICIO 3 =

abrindo na porta 5500 funciona normalmente



testando a porta 5501



EXERCICIO 4 =

Html:

```
<!DOCTYPE html>

<html lang="pt-BR">



<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width,
initial-scale=1.0">

  <title>Consulta de usuários</title>

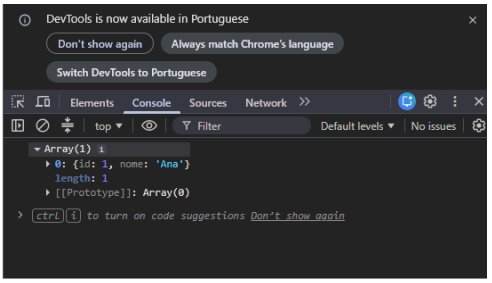
</head>
```

```
<body>
  <h1>Usuários</h1>
  <p>O resultado também aparece no console do navegador (F12).</p>
  <pre id="resultado">Consultando a API...</pre>
  <script>
    fetch('http://localhost:3000/usuarios')
      .then(res => res.json())
      .then(dados => {
        console.log(dados);
        document.getElementById('resultado').textContent =
          JSON.stringify(dados, null, 2);
      })
      .catch(erro => {
        console.error(erro);
        document.getElementById('resultado').textContent =
          'Não foi possível consultar a API.';
      });
  </script>
</body>


</html>
```

Teste de CORS

Aperte F12 e veja a aba Console.



EXERCICIO 5=

Trecho ultilizado:

```
app.use(cors({
  origin: ['http://localhost:5500'],
  methods: ['GET']
}));
```

O get retorna os usuarios normalmente mas , uma tentativa de DELETE gerou um preflight e foi bloqueada pelo navegador, pois esse método não constava nos métodos informados pelo servidor